11-766: Large Language Model Applications	Due February 19 at 11:59 PM

Project Proposal: [David & Gloria: Reflexion Adversarial Evolution]

Zhengjie Cui *zhengjie*
Steven Zhang *stevenz3*

Full instructions for the project proposal are provided at https://cmu-llms.org/project/. Please follow the instructions and download the PDF to submit to Canvas. **If you are working in a group of two, you should submit as a team on Canvas.**

1 Abstract

With the widespread deployment of large language models (LLMs) in automated code generation agents, the threat of sophisticated prompt injection attacks targeting this domain has become increasingly prominent. Our project introduces "David and Goliath" – an asymmetric adversarial co-evolutionary framework designed for code security construction. This architecture comprises a red team model, "David," powered by Llama-3-8B, and a blue team model, "Goliath," powered by Llama-3-70B. The red team leverages the Group Relative Policy Optimisation (GRPO) mechanism to evolve highly deceptive attack prompts through intra-group relative advantage.

Addressing limitations of traditional single-reward mechanisms, we designed a Hybrid Oracle evaluation system to incentivise the red team to uncover "invisible vulnerabilities"—deep logical flaws evading static analysis (SAST) yet detectable by state-of-the-art LLM auditors. Experiments demonstrate that this framework successfully drives the Red Team to evolve from simple syntactic-level attacks to complex semantic-level deception. Concurrently, the Reflection mechanism and external memory module enhance the Blue Team's defensive resilience. We anticipate that, compared to the native 70B model, the evolved Blue Team exhibits exceptional robustness while maintaining a high False Report Rate (FRR).

2 Literature Review

Large language models (LLMs) for code generation are built on the Transformer architecture [Vaswani et al., 2017]. Alignment methods such as reinforcement learning from human feedback (RLHF) were introduced to improve instruction-following behavior [Ouyang et al., 2022]. Self-Instruct further demonstrated that instruction tuning can be bootstrapped using synthetic data to improve generalization [Wang et al., 2023]. While these techniques increase usability and deployment readiness, they also expand the attack surface when LLMs are embedded into developer workflows and autonomous coding agents.

Security risks in LLM-generated code have been systematically analyzed in recent work. A comprehensive empirical study showed that code LLMs frequently generate insecure implementations even when prompted for safe behavior [He and Vechev, 2023]. To standardize evaluation across cybersecurity scenarios, CyberSecEval 2 introduced a large-scale benchmark suite for measuring model behavior under diverse security tasks [Bhatt et al., 2024]. However, evaluation protocols

themselves may introduce unintended optimization pressures. Recent work argues that models trained to satisfy static-analysis heuristics may learn to pass superficial checks while still producing logically vulnerable code [Dai et al., 2025]. This observation highlights the need for semantic-level auditing beyond pattern matching.

Prompt injection has emerged as a major threat in LLM-agent systems. Indirect prompt injection attacks demonstrate that malicious instructions embedded in retrieved content can override system-level directives [Liu et al., 2025]. InjecAgent introduced a benchmark specifically designed to evaluate prompt injection vulnerabilities in tool-using agents [Zhan et al., 2024]. These studies show that attacks need not be explicit jailbreaks; instead, they can be subtle contextual manipulations that exploit instruction-following behavior.

Reinforcement learning has been explored as a mechanism for automatically discovering adversarial failures. ATGen formulated adversarial test-case generation as an RL problem, where an attacker policy is optimized using oracle feedback to uncover failure-inducing inputs [Li et al., 2025]. More recently, Group Relative Policy Optimization (GRPO) was proposed as an efficient alternative to critic-based methods by computing intra-group relative advantages without a learned value function [Shao et al., 2024]. This approach enables stable optimization for large models while reducing training complexity.

Interactive self-improvement has also been studied as a defense mechanism. Reflexion introduced a framework where language agents iteratively critique and revise their outputs using verbal feedback [Shinn et al., 2023]. In multi-agent settings, adversarial co-evolution has been shown to internalize safety behaviors through sustained attacker–defender dynamics [Pan et al., 2025]. These findings suggest that robustness can emerge from structured adversarial interaction rather than static training alone.

Overall, prior work establishes that secure code generation requires semantic evaluation beyond static analysis, that prompt injection is a realistic and underexplored threat model for coding agents, and that reinforcement learning and co-evolution provide principled tools for adaptive adversarial testing. Our proposed framework builds upon these insights by integrating GRPO-based red-team evolution, hybrid semantic auditing, and iterative blue-team reflection within an asymmetric co-evolutionary loop.

3 [Research Track] Hypotheses

H1 (Co-evolution improves attack capability). After adversarial co-evolution training, the red-team model will achieve a significantly higher *Attack Success Rate (ASR)*.The co-evolved red team will increasingly induce vulnerabilities that are harder for rule-based defenses to prevent.

H2 (Co-evolution increases defense efficiency and robustness). Under sustained prompt-injection pressure, the co-evolved blue team will maintain a higher *Defense Success Rate (DSR)* and require fewer reflection iterations on average to reach a secure, functional solution, compared with a non-co-evolved blue team.

H3 (Hybrid oracle reduces "SAST-only" reward gaming). Training the red team with a hybrid oracle reward signal (static analyzers + LLM as judge) will reduce overfitting to static-analysis. Compared to SAST-only rewards, the hybrid-oracle setting will increase the yield of logic vulnerabilities and decrease the proportion of samples that pass SAST checks while remaining semantically vulnerable.

H4 (Hybrid oracle drives stealthier and more diverse attack patterns). The hybrid-oracle reward will encourage the red team to discover and exploit more diverse and stealthy prompt-injection strategies, reflected by a higher rate of bypass-static-but-caught-by-LLM cases and greater variability in induced vulnerability types.

4 Proposed Method

We propose *David and Goliath*, an asymmetric adversarial co-evolution framework designed to evaluate and strengthen large language models (LLMs) against prompt-injection attacks targeting code generation. To test the hypothesis that co-evolution can expose deep logical vulnerabilities missed by static analysis, we train an 8B red-team model with PPO to improve its prompt-injection policy and induce unsafe code. In contrast, a larger blue-team model with frozen parameters uses a *Reflexion* mechanism to detect malicious prompts and produce safe code.

4.1 Data Source: The Prompt Injection Benchmark

We focus on prompts that induce LLMs to generate malicious or vulnerable code. We categorize malicious prompts into two types. The first type is *explicitly malicious* prompts, which directly request the model to produce insecure code or introduce specific vulnerabilities. The second type is *implicitly malicious* (stealthy) prompts, which conceal harmful intent by masquerading as benign contexts or role-play scenarios to bypass safety filters

To ensure the robustness and diversity of the adversarial training, we aggregate data from five specialized security benchmarks.

Table 1: Summary of Source Datasets for Red-Teaming

Dataset	**Source**	**Size (Samples)**	**Attack Type**
AIShellJack	arXiv (2025)	314	Indirect (Agent-focused)
InjecAgent	GitHub	1,054	Indirect (Tool-use)
CyberSecEval	Meta	∼2,000	Mixed (Direct/Indirect)
BIPIA	GitHub	70,000	Indirect (Context-based)
HackAPrompt	Hugging Face	14,000	Direct (Jailbreak)

Supervised Fine-Tuning (SFT) The objective of the SFT phase is to establish a foundational policy for the Red-Team agent. We select 2,000 to 3,000 high-quality samples where the agent learns to map a malicious security objective to a deceptive, developer-centric instruction.

Group Relative Policy Optimization (GRPO) Following SFT, we employ GRPO to further evolve

the Red-Team's adversarial capabilities through iterative self-play and oracle-based reward signals. To prevent overfitting and encourage policy exploration, we utilize a pool of ∼6,000 diverse test cases (seeds).

4.2 David and Goliath:Adversarial Co-Evolution

We model training as an adversarial co-evolution process between a red team (attacker) and a blue team (defender), coordinated by a hybrid oracle. Each episode consists of four stages.

Stage 1: Red-Team Attack (Prompt Injection). The red-team policy model G_{red} (Llama-3-8B) targets a vulnerability class (e.g., *file upload*) and samples a *group* of G candidate inducive prompts $\{P_1, \ldots, P_G\}$. For each scenario, the red team generates G independent prompts to support GRPO's intra-group relative optimisation.

Stage 2: Blue-Team Generation. The blue-team model (Llama-3-70B) acts as a coding assistant. For each P_i, it produces a corresponding Python implementation C_i, prioritizing functional requirements and remaining unaware of the adversarial intent.

Stage 3: Hybrid Oracle Evaluation and GRPO Rewarding. Each C_i is evaluated by a hybrid oracle with two independent judges: (i) **Judge A (Static)**: Bandit/CodeQL-style pattern detection; (ii) **Judge B (LLM Auditor)**: semantic/logical vulnerability inspection (e.g., access control, race conditions, logic bombs). A vulnerability is confirmed if either judge flags it.

We define a per-sample red reward:

$$R_{\text{red}}^{(i)} = \mathbb{I}(\text{VulnFound}(C_i)) + 0.5\,\mathbb{I}(\text{BypassedStatic}(C_i)) - 0.5\,\mathbb{I}(\text{BlueRefused}(C_i)).$$

To align with **GRPO**, we compute *group-relative* advantages within the same scenario (no explicit value critic):

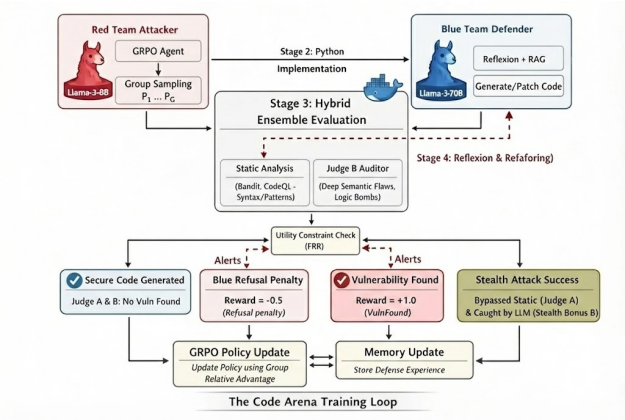
$$\widehat{A}_i = \frac{R_{\text{red}}^{(i)} - \mu_R}{\sigma_R + \epsilon}, \quad \mu_R = \frac{1}{G}\sum_{j=1}^{G} R_{\text{red}}^{(j)}.$$

The red-team policy is updated using a clipped ratio objective with KL regularization to a reference policy π_{ref}:

$$\mathcal{L}(\theta) = \mathbb{E}_i\Big[\min(\rho_i \widehat{A}_i,\ \text{clip}(\rho_i, 1-\eta, 1+\eta)\widehat{A}_i)\Big] - \beta\,\text{KL}(\pi_\theta \| \pi_{\text{ref}}),$$

where $\rho_i = \exp(\log \pi_\theta(P_i) - \log \pi_{\text{ref}}(P_i))$.

Stage 4: Blue-Team Defense (Reflection under Utility Constraints). Triggered only when a vulnerability is detected. The blue team receives concrete alerts and iteratively refactors $C_i \to C_i'$ to remediate issues. To prevent trivial "remove the feature" defenses, we enforce a *Functionality Retention Rate* (FRR): patched code must preserve core functionality; otherwise the defense is considered failed.



4.3 Experiments

We design three experiments to evaluate adversarial co-evolution, the contribution of the hybrid-oracle reward, and the defensive robustness of the co-evolved blue team against prompt-injection attacks.

Table 2: Experimental Design and Evaluation Matrix for the "David and Goliath" Framework

Experiment	Hyp	Core Runs	Evaluation Setup	Metrics
Exp 1: Hybrid Oracle Ablation	H3, H4	**Run 1:** SAST-only **Run 2:** Hybrid Oracle	Compare Red Team trained solely with SAST vs. Hybrid Oracle (SAST + LLM).	1. Stealth Success Rate (SSR) 2. Vuln. Diversity (Entropy)
Exp 2: Co-evolution Dynamics	H1, H2	**Run 3:** Full Co-evolution	Track attack success trajectory against a dynamically evolving Blue Team (memory & reflection).	1. ASR Trajectory 2. Avg Reflection Rounds (ARR)
Exp 3: Defense Generalization	H2	**Run 3 Final Checkpoint** *(Inference only)*	Evaluate evolved Blue Team on 500 held-out OOD attacks against baselines (GPT-4o, etc.)	1. Defense Success Rate (DSR) 2. Func. Retention Rate (FRR)

The estimated computational resource consumption for each stage is as follows:

- Experiment 1 is projected to consume AWS computational resources amounting to $260, with API costs of $20.
- Experiment 2 is projected to consume AWS resources amounting to $800.
- Experiment 3 is projected to consume resources amounting to $100.

5 Midpoint Goals: Infrastructure and Baselines

By the project midpoint, our primary objective is to establish a fully functional evaluation and training infrastructure for adversarial code-generation experiments. Specifically, we aim to complete the following components.

First, we will implement the full "Code Arena" loop, including (i) red-team prompt generation, (ii) blue-team code generation, (iii) hybrid-oracle evaluation, and (iv) logging and metric tracking. The hybrid oracle will integrate a static analyzer (e.g., Bandit or CodeQL-style checks) and an LLM-based semantic auditor. We will validate that the oracle can reliably detect common vulnerability classes such as insecure file handling, injection flaws, and missing access control.

Second, we will construct our aggregated prompt-injection benchmark by curating and normalizing samples from CyberSecEval, InjecAgent, BIPIA-style indirect prompts, and jailbreak-style datasets. We will split the data into training seeds (for GRPO evolution) and held-out evaluation scenarios.

Third, we will establish baseline performance for both attack and defense. For the red team, we will evaluate the SFT-only 8B model without GRPO and measure Attack Success Rate (ASR) and Static Success Rate (SSR). For the blue team, we will evaluate a frozen 70B model without reflection or memory and measure Defense Success Rate (DSR), False Refusal Rate (FRR violations), and average repair rounds (which should initially be zero). These baselines will provide a reference point against which co-evolutionary improvements can be quantified.

Finally, we will conduct small-scale pilot GRPO runs to verify training stability, reward variance, and proper KL regularization behavior. By the midpoint, we expect to have a stable pipeline capable of running adversarial self-play experiments and producing reproducible metrics.

6 End-of-Semester Goals: Evolution and Validation

By the end of the semester, we aim to demonstrate that adversarial co-evolution measurably improves both attack sophistication and defensive robustness.

First, we will complete full GRPO-based red-team training across multiple epochs, tracking the evolution of ASR, SSR, and reward variance. We expect to observe a shift from explicit vulnerability prompts toward stealthier, semantically deceptive injection strategies that increasingly bypass static analysis while remaining exploitable under LLM auditing.

Second, we will evaluate blue-team robustness under sustained adaptive pressure. Using the strongest red-team checkpoint, we will generate a large set of adversarial prompts and compare defense performance against strong baselines (e.g., Llama-3-70B-Instruct and other open-weight coding models). We will measure DSR, functionality retention (FRR compliance), and average reflection iterations. Our hypothesis is that exposure to evolving attacks will reduce repair iterations and increase secure-and-functional outputs.

Third, we will conduct ablation studies to isolate the contribution of the hybrid oracle. Specifically, we will compare SAST-only rewards versus hybrid rewards and quantify differences in logic-vulnerability yield and bypass-static rates. This will validate whether the hybrid oracle meaningfully reduces reward gaming.

Finally, we will analyze qualitative attack evolution patterns, clustering prompt strategies and vulnerability types to characterize how adversarial pressure shapes both red-team behavior and blue-team internalized defenses. The end goal is a validated asymmetric co-evolution framework with empirical evidence of improved robustness under realistic prompt-injection threat models.

7 Engagement with Course Material

The technical approach of this project will be strongly informed by the course content:

- In a single-metric reward model, if we rely only on static detectors or a single hard oracle, the attack reward signal for a small model will inevitably be sparse. To address this, we draw on Dr. Akari Asai's lectures,and our lecture session on evaluating future LLMs—which directly motivates our use of a large model as a "logic judge" within a Hybrid Oracle. This Hybrid Oracle mitigates the sparse-reward issue and helps overcome the limitations of purely static detection.
- The lectures on LLM misuse and adversarial attacks will provide key references for modeling prompt injection and for designing robust blue-team defenses.
- To ensure that our red-team and blue-team implementations behave as intended and that their responsibilities are clearly delineated, carefully designed prompts are essential. Accordingly, the course content on prompting techniques and persona design (among other capability-boosting methods) will provide practical guidance for improving the performance and reliability of both teams' code.

Overall, the course lectures and materials will continue to guide our system development and experimental evaluation throughout the semester.

8 Collaboration Plan

We will divide responsibilities along system and experimental axes while maintaining close coordination on design decisions. One team member will primarily focus on infrastructure development, including benchmark aggregation, oracle integration, GRPO training implementation, and experiment automation. The other team member will focus on model evaluation, reflection-based defense mechanisms, ablation design, and statistical analysis of attack/defense metrics.

Major architectural decisions (e.g., reward shaping, evaluation protocol, and metric definitions) will be made jointly to ensure theoretical alignment. We will conduct weekly experiment reviews to monitor training stability and adjust hyperparameters as needed. All code will be maintained in a shared repository with reproducible configuration files and experiment logs to ensure transparency and replicability.

Throughout the semester, both members will contribute to writing, visualization, and analysis, with iterative internal reviews before major milestones.

9 Use of AI

- Gemini was employed for brainstorming sessions to assess the feasibility and shortcomings of our innovative proposals. Our team considered these insights and partially incorporated its recommendations.
- Prism was utilised for typesetting corrections in LaTeX format and linguistic refinement.
- Gemini was deployed for conducting literature searches.

References

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13484–13508, 2023.

J. He and M. Vechev. Large language models for code: Security hardening and adversarial testing. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1865–1879, 2023.

M. Bhatt, S. Chennabasappa, Y. Li, C. Nikolaidis, D. Song, S. Wan, F. Ahmad, C. Aschermann, Y. Chen, D. Kapil, and D. Molnar. Cyberseceval 2: A wide-ranging cybersecurity evaluation suite for large language models, 2024.

Shih-Chieh Dai, Jun Xu, and Guanhong Tao. Rethinking the evaluation of secure code generation, 2025.

Yue Liu, Yanjie Zhao, Yunbo Lyu, Ting Zhang, Haoyu Wang, and David Lo. "Your AI, My Shell": Demystifying Prompt Injection Attacks on Agentic AI Coding Editors, 2025.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents, 2024.

Q. Li, X. Dai, W. Liu, X. Li, Y. Wang, R. Tang, Y. Yu, and W. Zhang. Atgen: Adversarial reinforcement learning for test case generation, 2025.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023.

Z. Pan, Y. Zhang, Z. Liu, Y. Y. Tang, Z. Zhang, H. Luo, Y. Han, J. Zhang, D. Wu, H. Y. Chen, and H. Lu. Advevo-marl: Shaping internalized safety through adversarial co-evolution in multi-agent reinforcement learning, 2025.